

SemCache: Semantic-Aware Cache Sharing for Efficient Multi-User LoRA-Adapted LLM Inference at the Edge

Tao Ren^{*}, Yiming Yao[†], Zheyuan Hu[†], Jianwei Niu^{†‡§}

^{*} State Key Laboratory of Intelligent Game, Institute of Software Chinese Academy of Sciences, University of Chinese Academy of Sciences, Beijing, China

[†] State Key Laboratory of Virtual Reality Technology and Systems, School of Computer Science and Engineering, Beihang University, Beijing, China; [‡] Zhongguancun Laboratory, Beijing, China; [§] Hangzhou Innovation Institute, Beihang University, Zhejiang Key Laboratory of Industrial Big Data and Robot Intelligent Systems, Hangzhou, China
Email: rentao22@iscas.ac.cn, {yaoyiming, huzheyuan18, niujianwei}@buaa.edu.cn

Abstract—Large language models (LLMs) enable personalized applications through parameter-efficient techniques like Low-Rank Adaptation (LoRA). Deploying LoRA-adapted LLMs faces a critical dilemma: resource-constrained user devices (UDs) are hard to host full-scale LLMs, while centralized edge deployments of all LoRA adapters incur huge storage and privacy risks. We propose *EdgeLoRA*, a collaborative framework where UD retain private LoRA adapters locally while offloading base model computations to edge servers (ESs), enhancing privacy and reducing storage overhead. However, *EdgeLoRA* introduces latency inefficiency from multi-round ES-UD interactions and memory overhead from redundant per-user key-value (KV) caching. To address this, we design *SemCache*, a semantic-aware caching mechanism that exploits hierarchical redundancies. *SemCache* first clusters user queries into coarse-grained intents (e.g., health, economic) via lightweight textual encoders, then identifies reusable fine-grained subsequences (e.g., “feel pain in”) within clusters. A hybrid-indexed global cache (cluster index + token position) enables cross-user KV reuse, while an adaptive policy optimizes cache utility via frequency, recency, and semantic impact metrics. Integrated with *EdgeLoRA*, *SemCache* reduces memory cost up to 10.8 $\times$ and achieves 2.11 $\times$ latency speedup per query via efficient token reuse, offering a scalable, privacy-preserving solution for efficient LoRA-adapted LLM inference at the edge.

Index Terms—LLM, Edge Inference, LoRA, Cache Share

I. INTRODUCTION

Large language models (LLMs) have shown remarkable capabilities in tasks such as question-answering, commonsense reasoning, and instruction following [1], [2]. These models are typically deployed on cloud or edge servers, allowing end-user devices to access their services via networks. However, such generic deployments struggle to meet personalized needs, as individuals often require LLMs tailored to their private data (e.g., healthcare records, shopping behaviors) to deliver domain-specific or context-aware assistance [3]–[5]. To bridge this gap, parameter-efficient fine-tuning (PEFT) methods, such as Low-Rank Adaptation (LoRA), have gained prominence for customizing pre-trained LLMs for personal tasks [6]–[8].

LoRA injects task-specific knowledge into a base LLM by updating low-rank matrices (*LoRA adapters*) while keeping the *base model* parameters frozen [9]. While effective, deploying

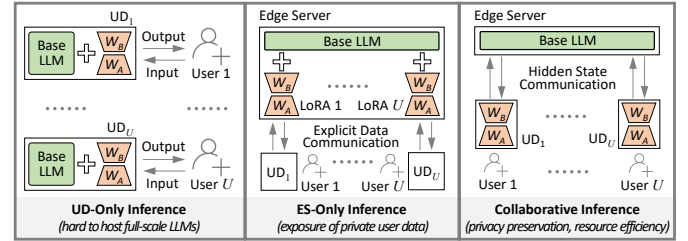


Fig. 1: LoRA-adapted LLM inference patterns in edge networks: (Left) UD-only inference (full local computation), faces resource constraints for hosting full-scale LLMs; (Middle) ES-only centralized inference (shared base model + adapters), risks privacy leakage of private user data and parameters; (Right) *EdgeLoRA* collaborative inference (UD-local adapters + ES-hosted base model), balances privacy preservation and resource efficiency through cross-user base model sharing.

LoRA-adapted personalized LLMs creates a critical dilemma: *resource-constrained end devices* (e.g., personal computers, laptops, and IoTs) are hard to host full-scale LLMs locally [10], [11], while *centralized edge deployment* of all personalized models incurs huge storage overhead (e.g., 1K users requiring 1K LoRA-adapter copies). Furthermore, centralizing LoRA adapters on servers risks exposing sensitive user data, as adapter parameters may encode private behavioral patterns [12], [13].

Recent advancements in LoRA, such as federated LoRA [14], [15] and HetLoRA [16], optimize distributed adaptation through collaborative training or client-specific rank allocation. However, these efforts focus on parameter training and neglect dynamic coordination during multi-user inference. Meanwhile, edge collaborative inference frameworks (e.g., model partitioning [17], device parallelism [18], and activation caching [19]), target small DNNs or static LLM architectures, failing to support dynamically personalized parameters. While recent works like *Edge-LLM* [20] and *OT-GAH* [21] improve edge inference via quantization or batch scheduling, they isolate user-specific components, overlooking cross-user computation reuse. This oversight results in redundant processing and suboptimal resource utilization, hindering scalable multi-user LLM deployment.

In view of this, we propose *EdgeLoRA*, a collaborative inference framework between edge server (ES) and user devices (UDs), as illustrated in Fig. 1 (Right), that leverages the architectural synergy between LoRA adapters and base LLMs. Unlike UD-only

This work was supported in part by National Key R&D Program of China under Grant No. 2023YFB4503700, National Natural Science Foundation of China under Grant No. U23B2025, 62372027. (Corresponding author: Jianwei Niu.)

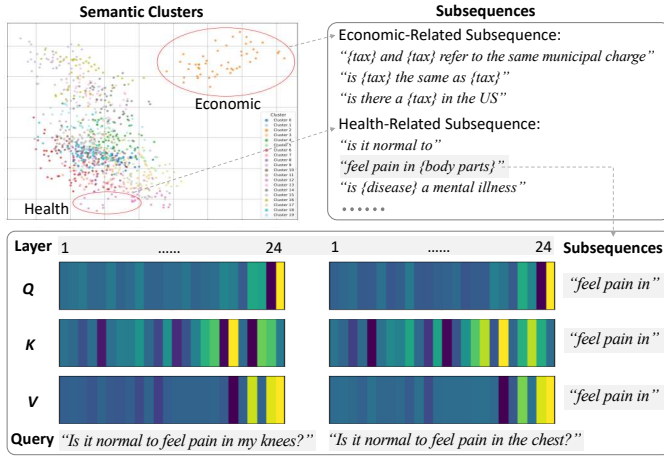


Fig. 2: Motivation experiment for cross-user Q/K/V reuse. Top: Semantic clusters and frequent subsequences in LoRA-adapted LLM queries. Bottom: Q/K/V similarity between subsequences from different queries belonging to the same cluster.

inference (Fig. 1, Left) which requires resource-constrained devices to host full LLMs locally, or ES-only inference (Fig. 1, Middle) which centralizes the base model and adapters on the server (risking privacy and scalability), EdgeLoRA enables multiple users to share a single base LLM on the ES while retaining personalized LoRA adapters exclusively on their own devices. This design ensures privacy preservation (no user raw data or LoRA adapters leave UD) and resource efficiency (centralized base model computation on the ES), enabling scalable personalized LLM inference at the edge [22], [23].

Despite its benefits, directly deploying EdgeLoRA also faces critical inefficiency challenges: (1) *Latency Inefficiency*. While edge networks reduce communication latency compared to the cloud, multi-round interactions between the ES (base model) and UD (LoRA adapters) during inference also introduce bottlenecks [24], since each token generation requires synchronizing multi-layer Q/K/V projections across ES-UDs. (2) *Memory Inefficiency*. The ES must cache independent key-value (KV) states per user due to LLMs' autoregressive decoding, leading to huge storage overhead. For instance, 10 users querying "Is it normal to feel pain in my knees" generate 10 redundant KV caches, consuming $10 \times$ GPU memory [25].

In fact, these inefficiencies stem from a missed opportunity: most user queries across different UD fall into recurring intent categories (e.g., health, economic) [26], yet existing frameworks fail to reuse computation across them. This observation raises a pivotal question:

Can the edge server reuse Q/K/V caches across users by exploiting semantic similarities among their queries?

To answer the question, we conduct a preliminary experiment on the MultiWOZ dataset [27], as shown in Fig. 2. We semantically cluster user queries, and **observe that** most queries belong to 19 intent clusters, with each cluster containing at least 30 high-frequency subsequences. Moreover, we compare Q/K/V states of a subsequence from two queries belonging to the same cluster, e.g., the "feel pain in" subsequence from query 1 "Is it normal to feel pain in my knees?" and query 2 "Is it normal to feel pain

in the chest?", and **find that** the Q/K/V of a subsequence from one query in each transformer layer is highly similar to that of the same subsequence from another, implying the potential to reuse caches across users.

Motivated by these insights, we further propose **SemCache**, a semantic-aware Q/K/V caching mechanism that not only reduces heavy storage overheads, but also accelerates multi-user LoRA-adapted LLM inference. Specifically, SemCache introduces a two-tiered semantic reuse strategy: 1) *Coarse-Grained Intent Clustering*, dynamically groups user queries into high-level semantic categories (e.g., health, economic, weather) to enable reuse of projection computations across shared intents. 2) *Fine-Grained Token Subsequencing*, identifies reusable token subsequences (e.g., "feel pain in", "weather in") within each cluster, maximizing granular cache sharing while preserving contextual relevance. To orchestrate this hierarchy, SemCache further employs a *global cache pool* with unified indexing of Q/K/V blocks by intent (cluster index) and position (token range) to enable rapid lookups, and a *hybrid admission/eviction policy* — weighting access frequency, recency, semantic impact, and memory footprint — to optimize cache utility under tight resource constraints.

To our knowledge, *SemCache is the first framework to exploit hierarchical semantic redundancies in multi-user LoRA-adapted LLM edge inference*, achieving significant reductions in both storage overhead (up to $10.8 \times$) and end-to-end latency ($2.11 \times$ faster per-query inference). This work bridges the gap between efficient, privacy-preserving LLM personalization and resource-constrained edge environments, unlocking applications like smart assistants and IoT services. The main contributions are summarized as follows:

- We propose EdgeLoRA, a collaborative inference paradigm where user devices retain personalized LoRA adapters locally while sharing a base LLM hosted on the edge server, achieving both privacy preservation and resource efficiency.
- We further propose SemCache, a two-tier cache mechanism that leverages semantic similarity across user queries to reuse Q/K/V, mitigating both latency inefficiency and storage overhead.
- We validate the effectiveness of SemCache via both simulation and testbed experiments, and also provide revealing insights through various ablation studies and parameter analysis.

II. RELATED WORK

A. LLM Low-Rank Adaptation

LoRA has emerged as a key technique for parameter-efficient fine-tuning of LLMs [28]. By freezing the base model and updating only low-rank matrices to inject task-specific knowledge, LoRA significantly reduces memory and computational costs compared to full-model fine-tuning [9]. Subsequent works have extended LoRA to multi-task settings [29], [30] and dynamic rank allocation [31], [32], further optimizing adaptation efficiency. Recent efforts also explore federated LoRA frameworks [14], [15], [33], where decentralized devices collaboratively train low-rank parameters under differential privacy. Additionally, HetLoRA [16] addresses edge-device heterogeneity through client-specific ranks and sparsity-weighted aggregation, optimizing convergence under resource limitations. However, these efforts focus on

training optimization, either centralizing parameters on servers (risking privacy) or assuming full-scale on-device deployments (ignoring resource constraints). Crucially, collaborative inference coordination for multi-user LoRA-adapted LLMs at the edge is neglected, leaving a critical research gap.

B. Distributed Edge Inference

Distributed edge inference aims to mitigate cloud latency and bandwidth bottlenecks [34]. Prior works optimize model partitioning [17], [35], [36], pruning [37], device parallelism [18], [38], [39], and lightweight activation caching [19], but primarily target small DNNs. Recent advances address large models via dynamic KV caching [26] and context-aware request batching [40], while personalized approaches like EdgeServe [41] cache user-specific model variants on ES, and Edge-LLM [20]/OT-GAH [21] optimize edge LLM inference via adaptive quantization or batch scheduling, both of which can be seen as ES-Only Inference. However, these works often focus on small DNN models or fixed large model architectures, failing to adapt to personalized model parameters. Critically, they treat KV caches as user-isolated, neglecting semantic overlaps across queries — leading to redundant storage and suboptimal resource utilization.

III. SYSTEM DESIGN OF EDGELORA

A. Edge and User Components

EdgeLoRA is deployed across ES and UD, as shown in the top and bottom parts of Fig. 3, respectively.

1) Components in ES host the base model's intermediate transformer layers $\ell_1 \rightarrow \ell_L$, each containing multi-head attention (MHA) and feed-forward network (FFN) parameters

$$W_{\text{MHA}} = \{W_Q^\ell, W_K^\ell, W_V^\ell, W_O^\ell\}, \quad W_{\text{FFN}} = \{W_{f1}, W_{f2}\},$$

where $W_Q^\ell, W_K^\ell, W_V^\ell \in \mathbb{R}^{d \times d_h}$ are query, key and value projection matrices for each head, $W_O^\ell \in \mathbb{R}^{d \times d}$ is the output projection, and $W_{f1} \in \mathbb{R}^{d \times d_f}$, $W_{f2} \in \mathbb{R}^{d_f \times d}$ are FFN linear transformations. Here, the head index for each $W_Q^\ell, W_K^\ell, W_V^\ell$ is omitted for simplicity, as do the biases for FFN.

2) Components in UD consist of three parts: LoRA adapters, input layer, and output layer.

LoRA Adapter: holds low-rank adaptation parameters¹ for Q, K and V across all heads and layers:

$$\Delta W_Q^{u,\ell}, \Delta W_K^{u,\ell}, \Delta W_V^{u,\ell}, \quad u \in \{1, 2, \dots, U\}, \quad \ell \in \{1, 2, \dots, L\},$$

where $\Delta W_Q^{u,\ell}, \Delta W_K^{u,\ell}, \Delta W_V^{u,\ell} \in \mathbb{R}^{d \times d_h}$ are factorized low-rank matrices, and U is the number of UDs.

Input Layer: holds token and positional encoders $E \in \mathbb{R}^{V \times d}$, $P \in \mathbb{R}^{n \times d}$, where n, V, d are the length of input (#tokens), vocabulary size, and hidden dimension, respectively.

Output Layer: holds the output logit projector $W_{\text{out}} \in \mathbb{R}^{d \times V}$, where the biases are omitted for simplicity.

B. Collaborative Inference

The collaborative inference consists of three stages:

1) Token Embedding (only UD)

¹ W_O and W_{FFN} can also be LoRA parameters, with $Q/K/V$ as typical cases.

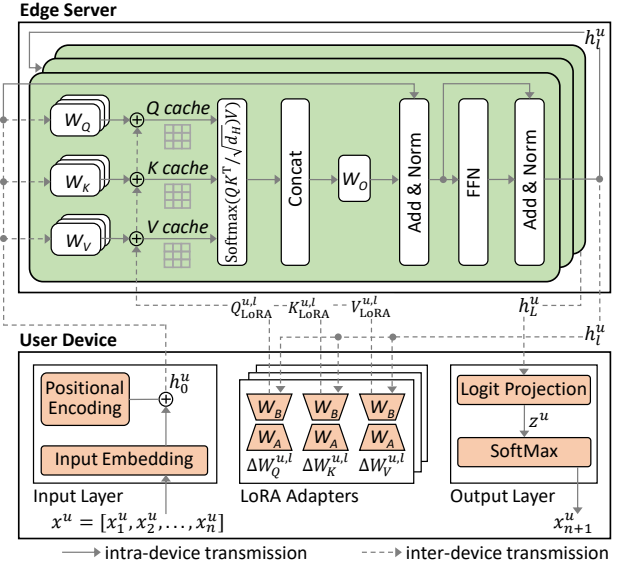


Fig. 3: The framework of EdgeLoRA, where the user device (holding the input layer, LoRA adapters, and output layer) and edge server (holding intermediate transformer layers) collaboratively perform LoRA inference, ensuring both privacy preservation and resource efficiency.

For an input query $x^u = [x_1^u, x_2^u, \dots, x_n^u]$ (where x_i^u is the i -th token) of UD u , token embedding and positional encoding are first performed locally:

$$h_0^u = E(x^u) + P(x^u), \quad (1)$$

where $h_0^u \in \mathbb{R}^{n \times d}$. Then, h_0^u is sent² to ES, for the iterative calculation of transformer layers from ℓ_1 to ℓ_L .

2) Iterative Transformers (ES and UD)

Multi-Head Attention (MHA): The ES and UD jointly compute Q, K, V for all heads $\{1, 2, \dots, H\}$:

$$\begin{aligned} Q &= Q_{\text{base}} + Q_{\text{LoRA}}, \quad Q_{\text{base}} = h_\ell^u W_Q^\ell, \quad Q_{\text{LoRA}} = h_\ell^u \Delta W_Q^{u,\ell}, \\ K &= K_{\text{base}} + K_{\text{LoRA}}, \quad K_{\text{base}} = h_\ell^u W_K^\ell, \quad K_{\text{LoRA}} = h_\ell^u \Delta W_K^{u,\ell}, \\ V &= V_{\text{base}} + V_{\text{LoRA}}, \quad V_{\text{base}} = h_\ell^u W_V^\ell, \quad V_{\text{LoRA}} = h_\ell^u \Delta W_V^{u,\ell}, \end{aligned} \quad (2)$$

and the attention is computed as:

$$\text{MHA} = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_H) W_O^\ell, \quad (3)$$

$$\text{head} = \text{softmax}(QK^\top / \sqrt{d_H}) V, \quad (4)$$

where $d_H = d/H$ is the input dimension of each head, and h_ℓ^u is equally divided for each head (omitting head index in h_ℓ^u for simplicity). After MHA, a residual connection and layer normalization are performed.

Note that the whole $Q/K/V$ projection computation implicitly involves a back-and-forth communication (as illustrated by the dashed line in Fig. 3): One is to send h_ℓ^u from ES to UD for LoRA projection computation, and the other is to return LoRA projections ($Q/K/V_{\text{LoRA}}$) back to ES.

Feed Forward Network (FFN): The FFN consists of two linear transformations with an activation σ in between:

$$\text{FFN} = \sigma(\text{MHA} W_{f1}) W_{f2},$$

²Lightweight differential privacy mechanisms could be applied to obfuscate sensitive intermediate embeddings during edge-device communication, enhancing privacy preservation without compromising computational efficiency.

followed by a residual connection and layer normalization, producing $h_{\ell+1}^u$ to the next layer.

3) Token Generation (only UD)

Logits Projection: ES sends the final hidden state h_L^u produced by the last layer L to the output layer in UD u , where the token logits z^u are calculated as follows:

$$z^u = h_L^u W_{\text{out}}. \quad (5)$$

Token Sampling: A softmax is performed to get the probability of each token as follows:

$$P(y_i^u) = \text{softmax}(z_i^u), \quad i \in \{1, 2, \dots, |V|\}, \quad (6)$$

which is used to sample a new token x_{n+1}^u for the user u .

Concatenating the new token x_{n+1}^u at the end of x^u , the above collaborative inference continues until meeting ‘EOS’.

IV. SEMCACHE

This section introduces SemCache, a semantic-aware caching framework that combines hierarchical intent clustering and a global cache pool to optimize multi-user LoRA-adapted inference.

A. Hierarchical Semantic Clustering

To exploit semantic redundancies across user requests, SemCache employs a two-tiered clustering strategy that groups queries hierarchically based on their semantic structure, as shown in Fig. 4.

1) Intent-Level Clustering

We group user queries into coarse-grained semantic categories (e.g., health, economic, weather) to enable high-level reuse of projection computations.

For an input token embedding h_0^u from user u , we adopt the TinyBERT (a distilled BERT variant) [42] to generate a compact semantic feature $e^u \in \mathbb{R}^{d_c}$:

$$e^u = \text{TinyBERT}(h_0^u), \quad (7)$$

which reduces computational overhead compared to full BERT, achieving a sub-millisecond latency per query on ES.

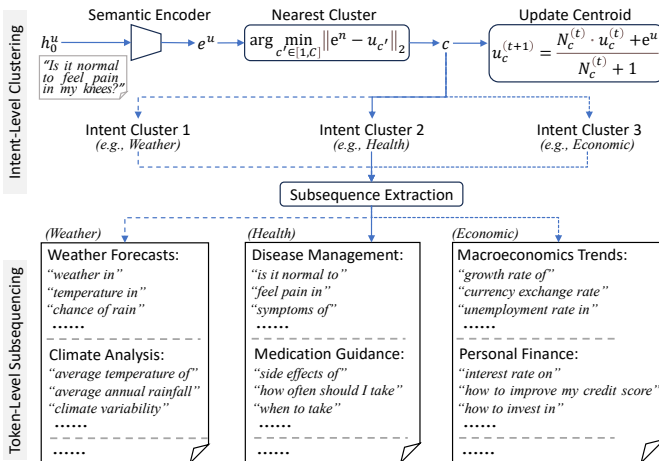


Fig. 4: Hierarchical semantic clustering workflow: (Top) Intent-Level Clustering, groups user queries into coarse-grained semantic categories (e.g., health, economic, weather) via lightweight embeddings; (Bottom) Token-Level Subsequencing, extracts fine-grained reusable patterns (e.g., “feel pain in”) to maximize cache reuse.

Semantic feature embeddings are grouped into C intent clusters with centroids $\{\mu_c\}_{c=1}^C$. The e^u is assigned a cluster:

$$c = \arg \min_{c' \in [1, C]} \|e^u - \mu_{c'}\|_2. \quad (8)$$

The centroids $\{\mu_c\}_{c=1}^C$ are updated incrementally to adapt to the distribution shift of queries:

$$\mu_c^{(t+1)} = \frac{N_c^{(t)} \cdot \mu_c^{(t)} + e^u}{N_c^{(t)} + 1}, \quad \text{if } e^u \rightarrow c, \quad (9)$$

where $N_c^{(t)}$ is the number of queries in cluster c at iteration t .

2) Token-Level Subsequencing

We further identify fine-grained reusable subsequences within each cluster (e.g., “feel pain in”) to maximize cache utility.

For each intent cluster c , we take a sliding window with fixed size w to extract overlapping token subsequences S_c from query embeddings h_0^u (excluding $P(x^u)$ in Eq. 1)

$$S_c = \{h_0^u[i : i + w] \mid h_0^u \in c, i \in [1, n - w]\}, \quad (10)$$

where $h_0^u[i : i + w]$ denotes a token subsequences from token i to $i + w - 1$ of h_0^u , and n is the number of tokens in a query.

B. Global Cache Pool

SemCache eliminates redundant storage by sharing $Q/K/V$ blocks across users via hierarchical indexing, as seen in Fig. 5.

1) Cache Structure

$Q/K/V$ cache blocks of the subsequence S_c^p are indexed by (c, p) , where c is the coarse-grained intent cluster (e.g., health), $p = [p_{\text{start}}, p_{\text{end}}]$ is the positional range of the fine-grained subsequence tokens (e.g., tokens 1-5 of “feel pain in”) in the cache pool of cluster c . So that all users querying “feel pain in {body parts}” share the same $Q/K/V$ cache for “feel pain in”, while “{body parts}” tokens are computed dynamically.

For a token subsequence S_c^p in cluster c with positional range p (spanning $w_p = p_{\text{end}} - p_{\text{start}} + 1$ tokens), the $Q_c^p, K_c^p, V_c^p \in \mathbb{R}^{w_p \times d}$ are concatenated into a single contiguous memory block $\mathcal{M}$ for coalesced access:

$$\mathcal{M}(c, p) = [Q_c^p K_c^p V_c^p] \in \mathbb{R}^{w_p \times 3d},$$

where Q_c^p, K_c^p, V_c^p are stored in row-major order, ensuring sequential access during attention computation.

2) Cache Admission and Eviction

To manage the global cache pool efficiently, SemCache employs a hybrid admission policy and eviction policy that balance frequency, recency, semantic impact, and memory footprint. The policy ensures high cache utility while minimizing computation and storage overhead.

For each cached memory block $\mathcal{M}(c, p)$, the following metrics are concerned:

- **Access Frequency** $\mathcal{F}(c, p)$: Number of times the subsequence has appeared in the latest queries (for admission score) or the block has been reused (for eviction score).
- **Cache Age** $\mathcal{A}(c, p)$: Time since last access.
- **Semantic Impact** $\mathcal{I}(c, p)$: Sum of attention weight norms for tokens in the block, i.e., $\mathcal{I} = \sum_{\ell} \sum_{t \in S_c} \|A_t^{\ell}\|_2$, where $A_t^{\ell} = \text{softmax}(Q_c^p [K_c^p]^{\top} / \sqrt{d_H})$ in layer ℓ and A_t^{ℓ} means each row (token) of A^{ℓ} .
- **Block Size** $\mathcal{S}(c, p)$: Memory footprint of the block.

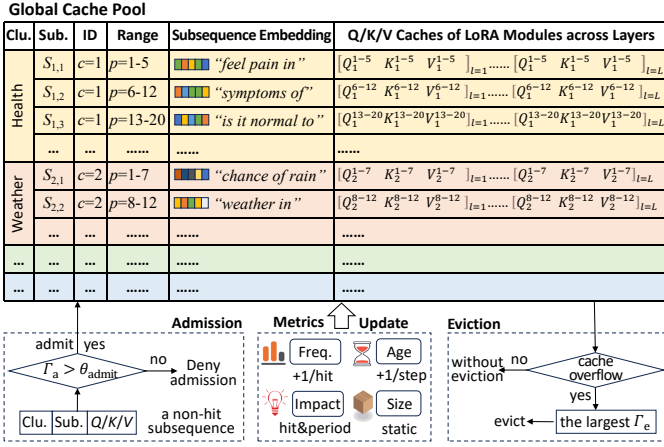


Fig. 5: Global cache pool architecture with hybrid admission and eviction policies. The cache is indexed by Cluster Index (e.g., health, economic) and Token Position Range (e.g., 1-5), enabling cross-user reuse of LoRA Q/K/V.

We decide whether to admit a new block into the pool based on the *admission score*³

$$\Gamma_a = \alpha \bar{\mathcal{F}} + \beta \bar{\mathcal{I}} + \delta(1 - \bar{\mathcal{S}}), \quad (11)$$

and evict from the pool based on the *eviction score*

$$\Gamma_e = \alpha' (1 - \bar{\mathcal{F}}) + \beta' (1 - \bar{\mathcal{I}}) + \gamma' \bar{\mathcal{A}} + \delta' \bar{\mathcal{S}}. \quad (12)$$

Here (c, p) is omitted for brevity, $\bar{\mathcal{F}} = \frac{\mathcal{F}}{\mathcal{F}_{\max}}$, $\bar{\mathcal{A}} = \frac{\mathcal{A}}{\mathcal{A}_{\max}}$, $\bar{\mathcal{I}} = \frac{\mathcal{I}}{\mathcal{I}_{\max}}$, $\bar{\mathcal{S}} = \frac{\mathcal{S}}{\mathcal{S}_{\max}}$ are the normalized frequency, age, impact, and size, respectively, $\mathcal{F}_{\max}, \mathcal{I}_{\max}, \mathcal{A}_{\max}, \mathcal{S}_{\max}$ are the maximum observed values across the cache, and α, β, δ and $\alpha', \beta', \gamma', \delta'$ are tunable weights satisfying $\alpha + \beta + \delta = 1$ and $\alpha' + \beta' + \gamma' + \delta' = 1$.

3) Metric-Specific Update Strategy

Among the four metrics, SemCache employs distinct update strategies to balance accuracy and overhead:

- Block Size $\mathcal{S}$: Static and computed once during admission; no further updates.
- Access Frequency $\mathcal{F}$ and Cache Age $\mathcal{A}$: Incremented every cache hit and time step, respectively.
- Semantic Impact $\mathcal{I}$: Recalculated dynamically to reflect evolving query contexts.

Concerning the following queries in the *health* cluster.

- Query1: “Is it normal to feel pain in my knees?” The subsequence S_c^p = “feel pain in” has high $\mathcal{I}(c, p)$ (directly influences output).
- Query2: “Is it serious to feel pain in my knees?” S_c^p has lower $\mathcal{I}(c, p)$ due to shifted focus (“serious”).

By dynamically recalculating $\mathcal{I}$, SemCache ensures that cached blocks reflect current semantic relevance while minimizing computational overhead.

However, recalculating $\mathcal{I}(c, p)$ for every inference step produces heavy computational loads, especially for a large hidden dimension d . To address computational overhead, SemCache adopts a tiered recalibration approach:

³A cache block is admitted into the global cache pool if $\Gamma_a > \theta_{\text{admit}}$. During pool overflow, the block with the highest Γ_e is evicted.

- *Cache Hit Updates (CHU)*: When a cached block $\mathcal{M}(c, p)$ is reused (cache hit), update $\mathcal{I}(c, p)$ using moving average

$$\mathcal{I}(c, p) = \rho \mathcal{I}(c, p) + (1 - \rho) \mathcal{I}_{\text{new}}(c, p),$$

where $\rho \in [0, 1)$ weights historical impact, and $\mathcal{I}_{\text{new}}$ is derived from the current query’s attention weights.

- *Periodic Batch Recalculation (PBR)*: During low query-load periods, recalculate $\mathcal{I}(c, p)$ for all blocks in cluster c using the average attention norms from the most recent Λ queries in that cluster,

$$\mathcal{I}(c, p) = \frac{\sum_{q \in \mathcal{Q}_c^\Lambda} \mathbb{I}_q(c, p) \cdot A_q(c, p)}{\sum_{q \in \mathcal{Q}_c^\Lambda} \mathbb{I}_q(c, p)}, \quad (13)$$

where $\mathcal{Q}_c^\Lambda$ is the set of the most recent Λ queries in cluster c , $\mathbb{I}_q(c, p)$ is the indicator function (1 if query q contains subsequence S_c^p , 0 otherwise), and $A_q(c, p)$ is the attention weight norm for S_c^p in q .

C. Integrating SemCache with EdgeLoRA

With the above, we bridge hierarchical clustering and cache operations to maximize reuse.

First, for an input h_0^u from user u , we generate an intent-level sentence embedding e^u , and assign it to cluster c . Consequently, we extract subsequences S_c and match them to cached positional ranges p .

Then, for cached non-overlapping subsequences S_c^p , fetch K_c^p, Q_c^p, V_c^p from cache memory in batches:

$$K_{\text{reuse}} = \bigcup_p K_c^p, Q_{\text{reuse}} = \bigcup_p Q_c^p, V_{\text{reuse}} = \bigcup_p V_c^p. \quad (14)$$

Next, we compute $Q_{\text{new}}, K_{\text{new}}, V_{\text{new}}$ matrices for unmatched tokens using base model weights W_Q, W_L, W_V and LoRA adapter weights $\Delta W_Q^u, \Delta W_L^u, \Delta W_V^u$.

Last, we combine reused Q/K/V blocks with dynamically computed tokens and perform MHA operations,

$$\text{head} = \text{softmax}([Q_{\text{reuse}}, Q_{\text{new}}][K_{\text{reuse}}, K_{\text{new}}]^T / \sqrt{d_H})[V_{\text{reuse}}, V_{\text{new}}].$$

The whole process is detailed in Algorithm 1. Note that only the latest one token is matched to retrieve reusable Q/K/V after generating the first token x_{n+1}^u rather than token subsequences extracted from the whole sentence (query with new tokens). We do not distinguish it from the initial query in the while loop of Algorithm 1 for brevity.

D. Latency Speedup Analysis

1) Without SemCache

For a query with n tokens, the computation cost for each transformer layer of the base model in the ES is

$$F_{\text{base}} = \underbrace{6nd^2 + 4n^2d + 2nd^2}_{\text{MHA}} + \underbrace{16nd^2 + 4nd}_{\text{FFN}} + \underbrace{12nd}_{\text{Add\&Norm}}. \quad (15)$$

The computation cost of each LoRA adapter (rank = r) in UD is

$$F_{\text{LoRA}} = 6ndr. \quad (16)$$

In addition, the communication overhead (measured as data size) between ES and UD for each transformer layer is

$$D_{\text{comm}} = \underbrace{nd}_{\text{ES} \rightarrow \text{UD (hidden states)}} + \underbrace{3nd}_{\text{UD} \rightarrow \text{ES (Q/K/V projections)}}. \quad (17)$$

Algorithm 1 SemCache-enhanced EdgeLoRA inference

Input: A query x^u from user u
Output: An answer y^u to user u
 Receive a query x^u in UD
while $x^u[-1] \neq \text{'EOS'}$ **do**
 Generate a token embedding h_0^u in UD ▷ Eq. 1
 Send h_0^u from UD to ES
 Generate a sentence embedding e^u ▷ Eq. 7
 Assign a cluster c ▷ Eq. 8
 Extract token subsequences S_c ▷ Eq. 10
 for $\ell = 1 \rightarrow L$ **do**
 Retrieve reusable $Q/K/V_{\text{reuse}}$ for S_c ▷ Eq. 14
 Compute $Q/K/V_{\text{new}}$ for unmatched tokens ▷ Eq. 2
 $Q/K/V \leftarrow [Q/K/V_{\text{reuse}}, Q/K/V_{\text{new}}]$
 Compute MHA(h_ℓ^u) ▷ Eqs. 3-4
 Compute FFN(h_ℓ^u) ▷ Eq. 5
 Produce $h_{\ell+1}^u$ to the next layer
 end for
 Send h_L^u from ES to UD
 Compute logits z^u ▷ Eq. 5
 Compute token probabilities $P(y_i^u)$ ▷ Eq. 6
 Generate a new token x_{n+1}^u
 Concatenate x_{n+1}^u to x^u
end while
return $y^u \leftarrow [x_{n+1}^u, \dots, \text{'EOS'}]$

The latency combining computation and communication bottlenecks for each query in a layer can be calculated as:

$$\mathcal{L} = \underbrace{\max\{6nd^2/f_{\text{ES}}, 6ndr/f_{\text{UD}} + 4nd/B\}}_{\text{max latency between } K/Q/V \text{ in ES and } \Delta K/Q/V \text{ in UD}} + \underbrace{(2nd^2 + 4n^2d + 16nd^2 + 4nd + 12nd)/f_{\text{ES}}}_{\text{the rest computation of base model in ES}}, \quad (18)$$

where f_{ES} , f_{UD} are ES, UD computational capacities, respectively, and B is the transmission rate between ES and UD.

2) With SemCache

Assuming n' tokens are reused from the cache, the latency reduction results from two parts: One is the $Q/K/V$ projection computation of the base model, i.e.,

$$\Delta F_{\text{base}} = 6n'd^2.$$

The other are the $\Delta Q/K/V$ projection computation and communication of the LoRA adapter, i.e.,

$$\Delta F_{\text{LoRA}} = 6n'dr, \quad \Delta D_{\text{comm}} = 4n'd.$$

The latency for each query in a layer with SemCache becomes

$$\mathcal{L}' = \max\{6(n-n')d^2/f_{\text{ES}}, 6(n-n')dr/f_{\text{UD}} + 4(n-n')d/B\} + (2nd^2 + 4n^2d + 16nd^2 + 4nd + 12nd)/f_{\text{ES}}. \quad (19)$$

3) Latency Speedup

Considering the significant computational capacity gap between ES and UD (i.e., $f_{\text{ES}} \gg f_{\text{UD}}$), the latency speedup can be approximated by ignoring ES computation terms (dominated by UD and communication latency) as follows:

$$\text{Speedup} \approx \frac{6ndr/f_{\text{UD}} + 4nd/B}{6(n-n')dr/f_{\text{UD}} + 4(n-n')d/B} = \frac{n}{n-n'}. \quad (20)$$

TABLE I: Hyperparameter Settings.

Hyperparameters	Values
Number of Users	50
LoRA Rank (r)	8
Intent Clusters (C)	20, 40, 30 for MultiWOZ, CoQA, SNIPS
Subsequence Length (w)	3
Cache Pool Size	20 GB
Admission Score Weights	$\alpha = 0.5, \beta = 0.3, \delta = 0.2$
Eviction Score Weights	$\alpha' = 0.4, \beta' = 0.3, \gamma' = 0.2, \delta' = 0.1$
Admission Threshold (θ_{admit})	0.3
Semantic Impact Update	$\rho = 0.8, \Lambda = 100$
Cluster Update Interval	100 queries

Assuming $n=512$ tokens (query length), $n'=212$ tokens,

$$\text{Speedup} \approx \frac{512}{512 - 212} \approx 1.71. \quad (21)$$

This indicates that each transformer layer can achieve a $1.71 \times$ speedup by reusing $n'/n=41\%$ tokens via SemCache, demonstrating significant efficiency gains for edge LLM inference.

V. EXPERIMENTS

A. Simulation Experiments

1) Experimental Setup

Hardware: The ES is simulated using an NVIDIA A100 GPU (80 GB VRAM) paired with Intel Xeon Gold 6338 CPUs. Each UD is simulated using four CPU cores (2.3 GHz) and 8GB RAM. The bandwidth between ES and UD is set to 200 Mbps.

LLM and LoRA: We employ three different LLMs, namely GPT2-Large (FP32), LLaMA-7B (INT4), and OPT-6.7B (FP16), to demonstrate the applicability of SemCache in mainstream LLMs. LoRA adapters are applied to the $Q/K/V$ matrices using Hugging Face PEFT [43].

Datasets: We adopt and adjust queries from the MultiWOZ [27], CoQA [44], and SNIPS [45] datasets for LoRA-adapted inference tasks. These tasks include multi-domain personalized queries in categories such as economic, health, and weather.

Baselines: We compare EdgeLoRA with two types of baselines, as shown in the left and middle of Fig. 1: **UD-Only** Inference and **ES-Only** Inference (representing EdgeServe [41], Edge-LLM [20] and OT-GAH [21]). We also evaluate a Frequency-Based Cache (**FBC**) approach combined with a least-recently-used policy for $Q/K/V$ caching without semantic awareness. Here, *ES-Only* serves as the reference benchmark for performance, disregarding privacy constraints, while UD-Only reflects a more practical deployment on constrained devices using smaller LLMs (GPT2-Large and LLaMA-7B (INT4)).

HyperParameters: Unless otherwise specified, hyperparameters (via grid search), are summarized in Table I.

2) Main Results

We evaluate SemCache on GPT2, LLaMA, and OPT models, w.r.t. system latency, memory cost, and answer accuracy⁴. Table II presents the results across all datasets.

Latency Speedup: SemCache substantially reduces system latency by enabling efficient cross-user $Q/K/V$ projection reuse. It achieves $2.11 \times$ and $2.07 \times$ speedup over UD-Only for GPT2 and LLaMA, respectively, while maintaining comparable

⁴Here and afterwards, the system latency and memory are abbreviated as Sys. Lat. and Mem., and the answer accuracy is denoted by BLEU score.

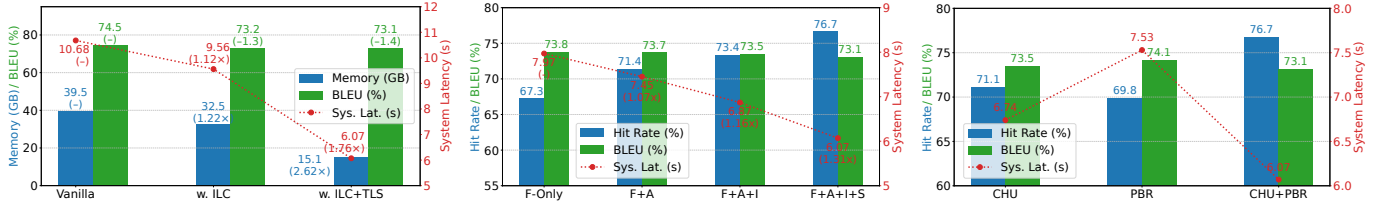


Fig. 6: Ablation Studies. Left: Semantic Reuse Granularity; Middle: Cache Management Metrics; Right: Semantic-Impact Update Strategies.

latency to ES-Only on OPT. Across all models, SemCache consistently outperforms the FBC baseline in latency.

Memory Reduction: SemCache significantly lowers memory usage to 23.9 GB for GPT2 and 25.1 GB for LLaMA, corresponding to 9.2 $\times$ and 10.8 $\times$ reductions relative to UD-Only. Compared to ES-Only, SemCache further reduces memory consumption while also enhancing privacy, supporting large-scale, privacy-preserving LoRA-based inference.

Answer Accuracy: SemCache maintains high answer quality, achieving BLEU scores of 71.9 and 72.5 for GPT2 and LLaMA, that match all baselines. For OPT, SemCache achieves a BLEU of 73.1 that obviously exceeds FBC, demonstrating a clear positive effect on accuracy.

Overall, SemCache delivers notable latency improvements, substantial memory savings, and strong answer accuracy, enabling efficient and scalable multi-user LoRA inference.

TABLE II: Performance comparison of baselines and SemCache (Ours). Performance is averaged per-query across datasets.

Model	Method	Sys. Lat. (s)	Mem. (GB)	BLEU (Δ) ²
GPT2	UD-Only	6.15 (1.00 $\times$)	220.5 ¹ (9.2 $\times$)	72.4 (—)
	FBC	3.57 (1.72 $\times$)	23.9 (1.0 $\times$)	71.4 (−1.0)
	Ours	2.91 (2.11 $\times$)	23.9 (1.0 $\times$)	71.9 (−0.5)
	ES-Only	2.54 (2.42 $\times$)	24.5 (1.03 $\times$)	72.4 (0.0)
LLaMA	UD-Only ³	7.53 (1.00 $\times$)	270.1 ¹ (10.8 $\times$)	72.9 (—)
	FBC	4.85 (1.55 $\times$)	25.1 (1.00 $\times$)	72.4 (−0.5)
	Ours	3.64 (2.07 $\times$)	25.1 (1.00 $\times$)	72.5 (−0.4)
	ES-Only	3.16 (2.38 $\times$)	25.6 (1.02 $\times$)	72.9 (0.0)
OPT	FBC	6.94 (0.82 $\times$)	35.2 (1.00 $\times$)	72.3 (−1.1)
	Ours	6.07 (0.94 $\times$)	35.2 (1.00 $\times$)	73.1 (−0.3)
	ES-Only	5.71 (1.00 $\times$)	36.8 (1.05 $\times$)	73.4 (—)

¹ Memory includes the base model parameters, LoRA adapters, Q/K/V caches and hidden activations, aggregated across ES and UDs.

² BLEU Δ (%) is the relative improvement over UD-Only for GPT2 and LLaMA; over ES-Only for OPT.

³ UD-Only runs only GPT2 and LLaMA(INT4) due to resource constraint.

3) Ablation Study

To understand the contributions of key design choices in SemCache, we perform three groups of ablation studies⁵:

Semantic Reuse Granularity: We compare three reuse strategies in Fig. 6 (Left):

- Vanilla EdgeLoRA: No cache; Q_{LoRA} , K_{LoRA} , V_{LoRA} are recomputed for each query.
- w. ILC: Reuses entire sentences via intent-level clustering.
- w. ILC+TLS: Full SemCache with fine-grained token-level subsequence caching.

Vanilla sees the highest latency and memory due to redundant query processing. ILC offers moderate improvements (1.12 $\times$

⁵ Due to space limits, we present the results on OPT-6.7B(FP16) here and afterwards.

faster, 1.22 $\times$ less memory) via sentence reuse. Full SemCache yields the best (1.76 $\times$ speedup responding to the speed analysis in Section IV-D, 2.62 $\times$ memory savings) by leveraging finer-grained overlap. BLEU drops slightly (≤ 1.4) due to approximation, but the tradeoff remains minimal, affirming its efficiency.

Cache Management Metrics: We evaluate four policies in Fig. 6 (Middle) with different combinations of metrics:

- F-Only: Only access frequency ($\mathcal{F}$) for admission/eviction.
- F+A: Combines $\mathcal{F}$ and cache age ($\mathcal{A}$).
- F+A+I: Adds semantic impact ($\mathcal{I}$) to $\mathcal{F} + \mathcal{A}$
- F+A+I+S: Incorporates $\mathcal{F}$, $\mathcal{A}$, $\mathcal{I}$ and block size ($\mathcal{S}$).

Under a fixed memory budget, F+A+I+S achieves the best performance (76.7% hit rate, 1.31 $\times$ speedup), with only a minor BLEU drop (0.7), validating the benefit of multi-factor reuse prioritization. F-Only performs worst (67.3% hit rate, 7.97 s latency), lacking semantic cues. F+A improves reuse quality (71.4%), and F+A+I further enhances hit rate (73.4%) by incorporating semantic alignment.

Impact Update Strategies: SemCache maintains per-cache-entry semantic impact $\mathcal{I}$ scores to reflect usage importance. We compare the following update strategies in Fig. 6 (Right):

- CHU (Cache Hit Update): update $\mathcal{I}$ only when cache hit.
- PBR (Periodic Batch Recalculation): refresh scores periodically based on global access patterns.
- CHU+PBR: the hybrid strategy combining both.

CHU and PBR yield moderate hit rates (71.1% and 69.8%), with CHU benefiting from its responsiveness to usage patterns. The hybrid CHU+PBR achieves the best results (76.7% hit rate, 6.07s latency), showing the complementarity of local and global updates. BLEU drop remains minimal (≤ 1), validating the tradeoff.

4) Method Parameter Analysis

Intent Clustering Cardinality: Table III shows the impact of clustering cardinality $C \in \{10, 20, 30, 40, 50\}$ on MultiWOZ, CoQA, and SNIPS. On MultiWOZ, smaller C (e.g., 20) yields high hit rates and low latency due to repetitive intents favoring coarse reuse. CoQA prefers larger C (40), capturing finer semantics despite reduced reuse. SNIPS performs best at $C = 30$, balancing reuse efficiency and semantic precision. These indicate that optimal C is dataset-dependent, shaped by intent diversity and reuse granularity.

Subsequence Window Size: Table IV shows that small windows (e.g., $w = 1$) reduce hit rate and increase latency across datasets due to fragmented reuse and limited semantic coverage. Larger windows (e.g., $w = 6$) lower hit rates by restricting reuse opportunities. A moderate size ($w = 3$) yields the best trade-off on all datasets, with SNIPS showing the peak hit rate (78.5%), and MultiWQZ achieving the lowest latency (5.09 s).

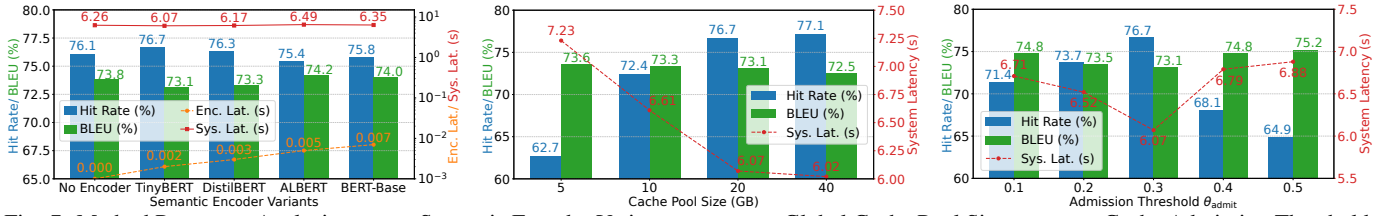


Fig. 7: Method Parameter Analysis. Left: Semantic Encoder Variants; Middle: Global Cache Pool Size; Right: Cache Admission Threshold.

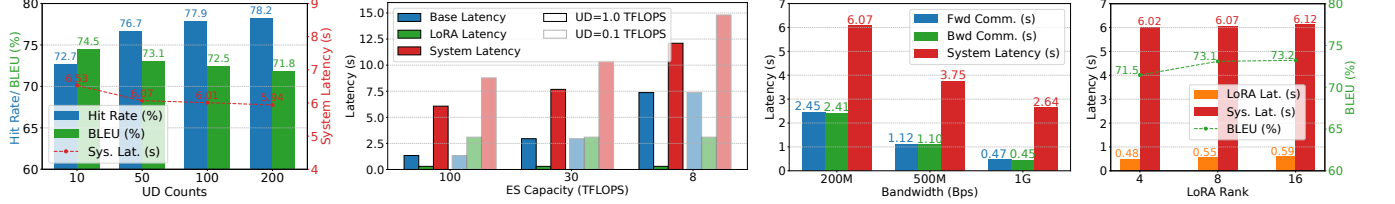


Fig. 8: System Parameter Analysis. First: UD Counts; Second: Computational Capability; Third: Network Bandwidth; Fourth: LoRA Rank.

TABLE III: Impact of Intent Clustering Cardinality

C	MultiWOZ			CoQA			SNIPS		
	H.	L.	B.	H.	L.	B.	H.	L.	B.
10	74.4	5.24	76.7	73.4	6.89	72.4	77.5	6.72	73.5
20	76.3	5.09	75.3	74.7	6.77	71.9	78.1	6.63	73.2
30	75.1	5.17	76.1	74.9	6.69	71.4	78.5	6.49	73.1
40	73.5	5.37	76.8	75.3	6.63	70.9	77.1	6.76	73.9
50	71.2	5.49	77.4	74.4	6.85	71.2	74.4	6.89	74.5

H.: Hit Rate (%), L.: Latency (s), B.: BLEU (%)

TABLE IV: Impact of Subsequence Window Size

w	MultiWOZ			CoQA			SNIPS		
	H.	L.	B.	H.	L.	B.	H.	L.	B.
1	74.1	5.35	76.2	73.2	6.87	72.1	73.8	6.74	75.1
2	74.5	5.21	75.9	74.5	6.78	71.7	75.8	6.58	74.2
3	76.3	5.09	75.3	75.3	6.63	70.9	78.5	6.49	73.1
4	74.5	5.24	75.5	73.2	6.86	71.9	75.3	6.62	73.9
5	73.8	5.44	76.8	72.6	6.94	72.6	74.4	6.69	74.4

This suggests $w = 3$ effectively captures reusable, semantically aligned subsequences while minimizing overhead.

Semantic Encoder Variants: We compare four encoders (TinyBERT, DistilBERT, ALBERT, and BERT-Base) in Fig. 7 (Left). TinyBERT achieves the best trade-off: lowest encoder (2ms) and system latency (6.07s), with solid hit rate (76.7%) and BLEU (73.1). BERT-Base yields higher accuracy (BLEU 74.0), but lower hit rate (75.8%) and higher system latency (6.35s). DistilBERT and ALBERT offer intermediate results.

Global Cache Pool Size: We report the impact of the global cache pool size in Fig. 7 (Middle). Performance improves steadily from 5 GB to 20 GB, with clear gains in hit rate and system latency. Beyond 20 GB, improvements saturate and latency slightly reduces, suggesting diminishing returns due to memory management overhead. 20 GB cache offers the best trade-off between reuse efficiency and memory cost.

Cache Admission Threshold: We analyze the impact of varying the cache admission threshold θ_{admit} in Fig. 7, Right. A low threshold (e.g., 0.1) admits many low-quality blocks, reducing efficiency. As θ_{admit} increases, hit rate and latency improve, peaking at $\theta_{\text{admit}} = 0.3$. Beyond that, stricter admission (e.g., 0.5) limits cache reuse, degrading latency performance. The optimal threshold of 0.3 consistently yields the highest hit rate

and lowest latency across datasets.

5) System Parameter Analysis

Scalability with UD Counts: As shown in Fig. 8 (First), increasing the number of UDs from 10 to 200 generally improves hit rate and reduces system latency, achieving the promising trade-off at 50 users with 76.7% hit rate and 6.07s latency. Beyond this point, marginal gains diminish due to increased cache contention under the given cache pool size. BLEU scores remain stable across all UDs, demonstrating that SemCache maintains inference quality at scale.

Impact of Computation Capability: Fig. 8 (Second) shows that ES capacity f_{ES} mainly affects base-model latency, while UD capacity f_{UD} dominates LoRA latency. System latency stays low when $f_{\text{UD}} = 1$ TFLOPS. Conversely, low UD capacity, i.e., $f_{\text{UD}} = 0.1$ TFLOPS, causes bottlenecks despite high f_{ES} , indicating that UD-side efficiency is key to overall responsiveness.

Sensitivity to Network Bandwidth: Fig. 8 (Third) shows that low bandwidth (200 Mbps) significantly increases communication latency, dominating system delay. As bandwidth rises, communication cost drops and system latency approaches the compute-bound baseline, highlighting the need for bandwidth-aware optimization in constrained networks.

Impact of LoRA Rank: Fig. 8 (Fourth) shows that as LoRA rank r grows from 4 to 16, both LoRA and system latency rise modestly, while BLEU climbs from 71.5 to 73.2. Notably, most accuracy improvement occurs between $r = 4$ and $r = 8$, with further gains becoming marginal at higher ranks. Thus, $r = 8$ strikes an optimal balance between efficiency and accuracy.

6) Method Overhead Analysis

Fig. 9 illustrates that increasing the PBR interval Λ from 100 to 1,000 queries sharply lowers its per-query overhead

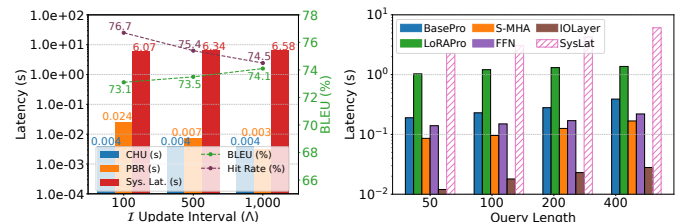


Fig. 9: Overhead Analysis.

Fig. 10: Latency Breakdown.

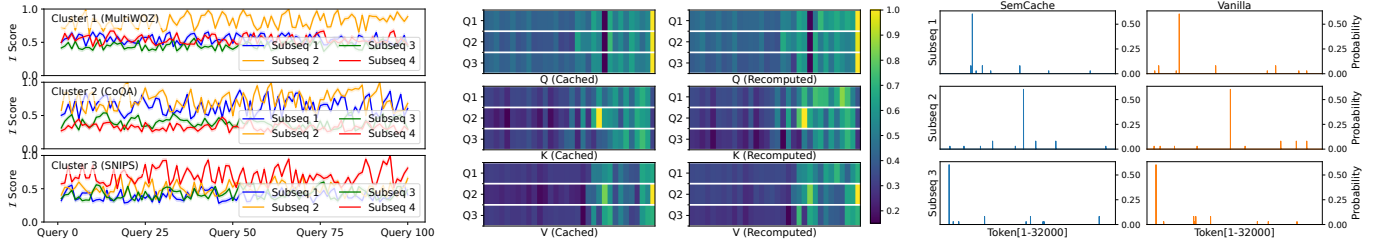


Fig. 11: Visualization Analysis. Left: Cache Semantic Impact; Middle: Cached and Recomputed Q/K/V; Right: Output Logit Distribution.

(0.024s to 0.003s), while CHU cost stays steady at 0.004s. This results in a small rise in system latency (from 6.07 s to 6.58 s) primarily due to the lower hit rate (from 76.7% to 74.5%). Overall, both update steps are lightweight, highlighting SemCache’s low-overhead design.

7) System Latency Breakdown

We analyze the average per-query latency of SemCache under varying input lengths ($\approx 50/100/200/400$ tokens). Fig. 10 shows a breakdown across major transformer components, including Base and LoRA Q/K/V projections, Softmax MHA, FFN, and I/O layers. As expected, latency grows with sequence length, with LoRA projection (including communication) remaining the dominant contributor due to remote adapter execution. Despite this, total system latency rises sublinearly with length, indicating effective reuse and pipeline efficiency from SemCache.

8) Visualization Analysis

We provide qualitative insights into SemCache by visualizing semantic dynamics and cache behavior across different datasets.

First, we select clusters from each dataset and visualize subsequences within each. As shown in Fig. 11 (Left), the semantic impact $\mathcal{I}$ of each subsequence varies slightly as queries evolve, indicating a distribution shift. However, the impact remains relatively stable over short time intervals, supporting the reuse assumption of SemCache.

Second, we compare the cached Q/K/V values of these subsequences in SemCache-enhanced EdgeLoRA with freshly recomputed ones from vanilla EdgeLoRA. As shown in Fig. 11 (Middle), cached and recomputed vectors remain highly aligned during early reuse phases and gradually diverge over time (Q1 $\rightarrow$ Q3), revealing that SemCache can retain representation fidelity for a meaningful window of reuse.

Third, we present case studies of queries, highlighting the matched subsequences and comparing the output logits between SemCache and vanilla EdgeLoRA. As shown in Fig. 11 (Right), the predictions remain consistent, while SemCache significantly reduces inference latency. These examples validate both the efficiency and effectiveness of the cache mechanism.

B. Testbed Experiments

As shown in Fig. 12, we employ NVIDIA RTX 4090 with 24 GB VRAM as ES and four personal computers (equipped with Core i9-9900K CPU processors) as 4 UD, along with a WiFi router with 300 Mbps bandwidth for high-speed data transfer between ES and UD. Other experimental setups are similar to those in simulation experiments.

In Table IV, the top-reference (ES-Only) generally achieves the lowest latency across all datasets and models (e.g., 1.67s

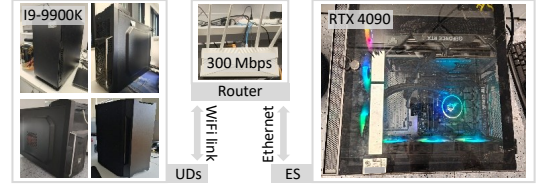


Fig. 12: Testbed experiment hardware.

TABLE V: Testbed Experimental Results (Latency: s, BLEU: %)

Model	Method	MultiWOZ		CoQA		SNIPS	
		Lat.	BLEU	Lat.	BLEU	Lat.	BLEU
GPT2	UD-Only	3.74	71.3	4.34	72.2	4.42	73.7
	FBC	2.01	70.1	2.51	71.2	2.57	72.9
	Ours	1.88	70.7	2.37	71.7	2.45	73.3
	ES-Only	1.67	71.3	2.14	72.2	2.21	73.7
LLaMA	UD-Only	3.90	75.3	4.43	71.3	4.55	72.1
	FBC	2.33	74.8	2.59	70.8	2.65	71.7
	Ours	2.13	74.8	2.46	70.9	2.51	71.8
	ES-Only	1.91	75.3	2.24	71.2	2.31	72.2
OPT	FBC	2.21	74.4	2.87	70.4	2.93	72.2
	Ours	2.09	75.3	2.69	70.9	2.75	73.1
	ES-Only	1.96	75.8	2.52	71.1	2.58	73.3

for GPT2 on MultiWOZ), while also maintaining high BLEU scores (e.g., 71.3 for GPT2 on MultiWOZ). UD-Only exhibits higher latency due to local computation (e.g., 3.74s for GPT2 on MultiWOZ), yet achieves comparable or slightly higher BLEU. FBC yields moderate latency and competitive BLEU (e.g., 2.01s latency and 70.1 BLEU for GPT2 on MultiWOZ). In comparison, SemCache provides a favorable balance, reducing latency to 1.88s for GPT2 on MultiWOZ, and achieving a BLEU score of 70.7. Compared to simulation, the latency reduction in testbed primarily results from the higher GPU and CPU frequencies. Similar trends are observed across CoQA and SNIPS, as well as for LLaMA and OPT models, where SemCache consistently outperforms UD-Only and FBC in latency while maintaining comparable answer quality, delivering low-latency inference with strong accuracy in practical scenarios.

VI. CONCLUSION

This paper introduces SemCache, a semantic-aware caching mechanism designed for collaborative multi-user EdgeLoRA inference. By leveraging a shared base LLM on ES and enabling cross-user semantic-aware LoRA cache reuse, SemCache improves both latency and cache efficiency at the resource-constrained edge. Our results show substantial performance gains, including up to $2.11\times$ latency speedup, while maintaining privacy. Future work will extend SemCache to accommodate heterogeneous LoRA adapters and UD in large-scale deployments.

REFERENCES

- [1] B. Min, H. Ross, E. Sulem *et al.*, “Recent advances in natural language processing via large pre-trained language models: A survey,” *ACM Comput. Surveys*, vol. 56, no. 2, pp. 1–40, 2023.
- [2] Z. Shao, Z. Yu, M. Wang *et al.*, “Prompting large language models with answer heuristics for knowledge-based visual question answering,” in *Proc. IEEE Conf. on Comput. Vis. and Pattern Recognit. (CVPR)*, Vancouver, Jun. 2023, pp. 14974–14983.
- [3] G. Jiang, M. Xu, S.-C. Zhu *et al.*, “Evaluating and inducing personality in pre-trained language models,” in *Proc. Annual Conf. on Neural Inf. Process. Syst. (NeurIPS)*, New Orleans, USA, Dec. 2023, pp. 10622–10643.
- [4] X. L. Dong, S. Moon, Y. E. Xu *et al.*, “Towards next-generation intelligent assistants leveraging LLM techniques,” in *Proc. ACM Knowl. Discov. and Data Min. (SIGKDD)*, New York, NY, USA, Aug. 2023, p. 5792–5793.
- [5] A. Salemi, S. Mysore, M. Bendersky *et al.*, “LaMP: When large language models meet personalization,” in *Proc. Annu. Meet. of the Assoc. for Comput. Linguist. (ACL)*, Bangkok, Thailand, Aug. 2024, pp. 7370–7392.
- [6] H. Liu, D. Tam, M. Muqeeth *et al.*, “Few-shot parameter-efficient finetuning is better and cheaper than in-context learning,” in *Proc. Annual Conf. on Neural Inf. Process. Syst. (NeurIPS)*, New Orleans, USA, Nov. 2022, pp. 1950–1965.
- [7] E. B. Zaken, Y. Goldberg, and S. Ravfogel, “Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models,” in *Proc. Annu. Meet. of the Assoc. for Comput. Linguist. (ACL)*, New York, NY, USA, May 2022, pp. 1–9.
- [8] N. Ding, Y. Qin, G. Yang *et al.*, “Parameter-efficient fine-tuning of large-scale pre-trained language models,” *Nature Mach. Intell.*, vol. 5, no. 3, pp. 220–235, 2023.
- [9] E. J. Hu, Y. Shen, P. Wallis *et al.*, “LoRA: Low-rank adaptation of large language models,” in *Proc. Int. Conf. on Learn. Represent. (ICLR)*, Virtual, Apr. 2022, pp. 1–13.
- [10] D. Xu, W. Yin, H. Zhang *et al.*, “EdgeLLM: Fast on-device LLM inference with speculative decoding,” *IEEE Trans. Mobile Comput.*, vol. 24, no. 4, pp. 3256–3273, 2025.
- [11] G. Qu, Q. Chen, W. Wei *et al.*, “Mobile edge intelligence for large language models: A contemporary survey,” *IEEE Commun. Surveys Tuts.*, pp. 1–20, 2025, early access.
- [12] L. Song and P. Mittal, “Systematic evaluation of privacy risks of machine learning models,” in *Proc. USENIX Security*, Virtual, Aug. 2021, pp. 2615–2632.
- [13] H. Chen, W. Deng, S. Yang *et al.*, “Towards edge general intelligence via large language models: Opportunities and challenges,” *IEEE Network*, pp. 1–10, 2025, early access.
- [14] H. Sun, H. Tian, W. Ni *et al.*, “Federated low-rank adaptation for large models fine-tuning over wireless networks,” *IEEE Trans. Wireless Commun.*, vol. 24, no. 1, pp. 659–675, 2025.
- [15] Z. Wang, Y. Zhou, Y. Shi *et al.*, “Federated low-rank adaptation for large language model fine-tuning over wireless networks,” in *Proc. GLOBECOM*, Cape Town, South Africa, Dec. 2024, pp. 3063–3068.
- [16] Y. J. Cho, L. Liu, Z. Xu *et al.*, “Heterogeneous LoRA for federated fine-tuning of on-device foundation models,” in *Proc. EMNLP*, Miami, Florida, USA, Nov. 2024, pp. 12903–12913.
- [17] T. Wang, T. Shi, X. Liu *et al.*, “Minimizing latency for multi-DNN inference on resource-limited CPU-only edge devices,” in *Proc. IEEE Int. Conf. on Comput. Commun. (INFOCOM)*, Vancouver, May 2024, pp. 2239–2248.
- [18] S. Ye, J. Du, L. Zeng *et al.*, “Galaxy: A resource-efficient collaborative edge AI system for in-situ transformer inference,” in *Proc. IEEE Int. Conf. on Comput. Commun. (INFOCOM)*, Vancouver, May 2024, pp. 1001–1010.
- [19] Y. Liu, H. Li, Y. Cheng *et al.*, “Cachegen: Kv cache compression and streaming for fast large language model serving,” in *Proc. SIGCOMM*, Sydney, Australia, Aug. 2024, pp. 38–56.
- [20] F. Cai, D. Yuan, Z. Yang *et al.*, “Edge-LLM: A collaborative framework for large language model serving in edge computing,” in *Proc. IEEE Int. Conf. on Web Svc. (ICWS)*, Bangkok, Thailand, Nov. 2024, pp. 799–809.
- [21] X. Zhang, J. Nie, Y. Huang *et al.*, “Beyond the cloud: Edge inference for generative large language models in wireless networks,” *IEEE Trans. Wireless Commun.*, vol. 24, no. 1, pp. 643–658, 2025.
- [22] D. Xu, T. Li, Y. Li *et al.*, “Edge intelligence: Empowering intelligence to the edge of network,” *Proc. of the IEEE*, vol. 109, no. 11, pp. 1778–1837, 2021.
- [23] Y. Tay, M. Dehghani, D. Bahri *et al.*, “Efficient transformers: A survey,” *ACM Comput. Surveys*, vol. 55, no. 6, pp. 1–28, 2022.
- [24] Y. Sheng, L. Zheng, B. Yuan *et al.*, “FlexGen: High-throughput generative inference of large language models with a single GPU,” in *Proc. Int. Conf. on Mach. Learn. (ICML)*, Hawaii, USA, Jul. 2023, pp. 31094–31116.
- [25] J. Li, J. Ma, Y. Miao *et al.*, “Verifiable semantic-aware ranked keyword search in cloud-assisted edge computing,” *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3591–3605, 2021.
- [26] W. Lee, J. Lee, J. Seo *et al.*, “InfiniGen: Efficient generative inference of large language models with dynamic KV cache management,” in *Proc. NSDI*, Apr. 2024, pp. 155–172.
- [27] P. Budzianowski, T.-H. Wen, B.-H. Tseng *et al.*, “MultiWOZ: A large-scale multi-domain Wizard-of-Oz dataset for task-oriented dialogue modelling,” in *Proc. EMNLP*, Brussels, Oct. 2018, pp. 5016–5026.
- [28] Z. Fu, H. Yang, A. M.-C. So *et al.*, “On the effectiveness of parameter-efficient fine-tuning,” in *Proc. AAAI Conf. on Artif. Intell. (AAAI)*, Washington DC, USA, Feb. 2023, pp. 12799–12807.
- [29] A. Agiza, M. Neseem, and S. Reda, “MTLoRA: Low-rank adaptation approach for efficient multi-task learning,” in *Proc. IEEE Conf. on Comput. Vis. and Pattern Recognit. (CVPR)*, Seattle, USA, Jun. 2024, pp. 16196–16205.
- [30] Y. Ma, Z. Liang, H. Dai *et al.*, “MoDULA: Mixture of domain-specific and universal LoRA for multi-task learning,” in *Proc. EMNLP*, Miami, Florida, USA, Nov. 2024, pp. 2758–2770.
- [31] Y. Mao, K. Huang, C. Guan *et al.*, “DoRA: Enhancing parameter-efficient fine-tuning with dynamic rank distribution,” in *Proc. Annu. Meet. of the Assoc. for Comput. Linguist. (ACL)*, Bangkok, Thailand, Aug. 2024, pp. 11662–11675.
- [32] M. Kim, S. Lee, W. Sung *et al.*, “RA-LoRA: Rank-adaptive parameter-efficient fine-tuning for accurate 2-bit quantized large language models,” in *Proc. Annu. Meet. of the Assoc. for Comput. Linguist. (ACL)*, Bangkok, Thailand, Aug. 2024, pp. 15773–15786.
- [33] Z. Wang, Z. Shen, Y. He *et al.*, “FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptations,” in *Proc. Annual Conf. on Neural Inf. Process. Syst. (NeurIPS)*, Vancouver, Dec. 2024, pp. 15773–15786.
- [34] M. M. H. Shuvo, S. K. Islam, J. Cheng *et al.*, “Efficient acceleration of deep learning inference on resource-constrained edge devices: A review,” *Proc. of the IEEE*, vol. 111, no. 1, pp. 42–91, 2022.
- [35] H. Li, X. Li, Q. Fan *et al.*, “Distributed DNN inference with fine-grained model partitioning in mobile edge computing networks,” *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9060–9074, 2024.
- [36] J.-A. Lim, J. Lee, J. Kwak *et al.*, “Cutting-edge inference: Dynamic DNN model partitioning and resource scaling for mobile AI,” *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 3300–3316, 2024.
- [37] L. Han, Z. Xiao, and Z. Li, “DTMM: Deploying TinyML models on extremely weak IoT devices with pruning,” in *Proc. IEEE Int. Conf. on Comput. Commun. (INFOCOM)*, Vancouver, May 2024, pp. 1999–2008.
- [38] C. Hu and B. Li, “Distributed inference with deep learning models across heterogeneous edge devices,” in *Proc. IEEE Int. Conf. on Comput. Commun. (INFOCOM)*, virtual, May 2022, pp. 330–339.
- [39] S. Ye, L. Zeng, X. Chu *et al.*, “Asteroid: Resource-efficient hybrid pipeline parallelism for collaborative DNN training on heterogeneous edge devices,” in *Proc. ACM Int. Conf. on Mobile Comput. and Netw. (MobiCom)*, New York, NY, USA, Nov. 2024, pp. 312–326.
- [40] X. Tan, H. Li, X. Xie *et al.*, “Reinforcement learning based online request scheduling framework for workload-adaptive edge deep learning inference,” *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13222–13239, 2024.
- [41] T. Guo, R. J. Walls, and S. S. Ogden, “EdgeServe: Efficient deep learning model caching at the edge,” in *Proc. ACM/IEEE Symp. on Edge Comput.*, Arlington, Virginia, USA, Nov. 2019, pp. 313–315.
- [42] X. Jiao, Y. Yin, L. Shang *et al.*, “Tinybert: Distilling bert for natural language understanding,” in *Proc. EMNLP*, Virtual, Nov. 2020, pp. 4163–4174.
- [43] H. Face, “PEFT: Parameter-efficient fine-tuning,” <https://github.com/huggingface/peft>, 2023, accessed: May 2, 2025.
- [44] S. Reddy, D. Chen, and C. D. Manning, “CoQA: A conversational question answering challenge,” *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 249–266, 2019.
- [45] A. Coucke, A. Saade, A. Ball *et al.*, “Snips voice platform: an embedded spoken language understanding system for private-by-design voice interfaces,” *ArXiv*, vol. abs/1805.10190, 2018.